Retrieval-Augmented Generation for Pakistani Law: Hybrid Search, Re-Ranking, and Automated Evaluation

SHAZAIN, 27115; SHAIKH MUHAMMAD SHARJEEL, 26932; SYED SHAYAN HUSSAIN, 26289

1 Introduction

This report details the design, implementation, and evaluation of a Retrieval-Augmented Generation (RAG) based Question-Answering system for the domain of Pakistani Law. The system answers legal queries by retrieving relevant statutory text and providing it as context to a Large Language Model (LLM).

Pakistani legal information is inaccessible to most citizens. Legal consultation is expensive, and general-purpose LLMs cannot serve as reliable alternatives due to knowledge gaps specific to Pakistani legislation. This gap is most pronounced in provisions modified under the Islamisation of Laws Ordinance (1979–1990), which replaced standard English legal terminology with Islamic legal terms throughout the Pakistan Penal Code. For example, "murder" was replaced by "qatl-i-amd", and standard punishments were replaced by the qisas and diyat framework. This terminology gap was empirically confirmed during system development: the query "What is the punishment for murder?" produced a cosine similarity of only 0.39 against the correct Section 302 chunk, confirming that general embedding models fail on this domain-specific vocabulary.

This project also simulates a real-world deployment scenario where an organisation cannot use proprietary cloud LLMs due to data privacy concerns or cost. An open-source LLM served via API is combined with a private document corpus through RAG, demonstrating a secure, cost-effective enterprise deployment pattern.

The intended users are: citizens seeking to understand their legal rights without legal fees, law students conducting preliminary statutory research, journalists verifying legal claims, small business owners requiring basic contract or company law guidance, and paralegals needing immediate statutory reference.

The architecture incorporates Hybrid Search (BM25 + dense retrieval), Cross-Encoder Re-Ranking, LLM-Based Query Rewriting for vocabulary bridging, and an Automated Evaluation pipeline using the LLM-as-a-Judge paradigm. The system is deployed as a publicly accessible web application.

2 Platform and Infrastructure Details

All experimentation and evaluation were conducted on Google Colaboratory with a T4 GPU runtime. Pinecone (free Serverless tier, AWS us-east-1) was used as the cloud vector database. The live web interface uses Gradio deployed on HuggingFace Spaces (free CPU tier).

The deployed application is publicly accessible at: https://huggingface.co/spaces/yaxsq/pakistani-law-rag.

LLM API Infrastructure. The Groq API free tier was used exclusively for the ablation study, providing access to `llama-3.3-70b-versatile` at high inference speed. However, the assignment technical stack specifies the HuggingFace Inference API for LLM generation. The production deployed application therefore uses `meta-llama/Llama-3.1-8B-Instruct` via the HuggingFace Inference API as the primary provider, with `llama-3.1-8b-instant` via Groq as an automatic fallback. When the HF provider returns any error (402 Payment Required, 410 Gone, or 429 Rate Limited), the system silently retries with Groq and updates the UI provider dropdown to reflect the switch. This dual-provider design satisfies the assignment infrastructure requirement while maintaining availability when HF free credits are exhausted.

Author's Contact Information: Shazain, 27115; Shaikh Muhammad Sharjeel, 26932; Syed Shayan Hussain, 26289.

Rate Limit Management. The Groq free tier imposes 500,000 tokens per day. A single ablation configuration consumed approximately 400,000–500,000 tokens. Evaluations were paced across multiple days with `time.sleep(3)` enforced between API calls. Config D required a full re-run after the first attempt was invalidated by mid-run rate limiting — all subsequent LLM calls returned empty responses, producing artificially zero faithfulness scores. Results in Table 5 reflect only the valid re-run.

HuggingFace Spaces Deployment. The interface status badge displayed "Building" for extended periods even when the application was fully operational. This is a known cosmetic issue with the free tier. Operational status was verified via backend logs and direct URL access. Free-tier containers spin down after approximately 15 minutes of inactivity. A cold start requires `startup()` to re-execute, adding 60–90 seconds to the first response after an idle period.

Secrets Management. API keys are stored as HuggingFace Space Secrets (`PINECONE_API_KEY`, `HF_TOKEN`, `GROQ_API_KEY`) and injected as environment variables at runtime. No keys appear in source code.

3 Data Details and Preprocessing

3.1 Corpus Collection

The corpus consists of official Pakistani government legal documents sourced from `pakistancode.gov.pk` and `commonlii.org`. All documents are public domain. Pakistani legal acts were selected because they represent a high-stakes domain where retrieval accuracy directly impacts user welfare, general LLMs underperform due to Islamised terminology, and the documents are sufficiently long and structured to produce a meaningful retrieval challenge.

Programmatic scraping failed due to inconsistent HTML structures and server restrictions on Colab IP ranges. Documents were manually copied from official sites, saved as `.txt` files, and loaded programmatically. Some metadata defaults to `year=0` as a cosmetic artifact of plain-text conversion; this does not affect retrieval or generation.

The corpus comprises 10 legal acts as detailed in Table 1, exceeding the requirement of 50–100 documents or 500+ chunks. Qualitative evaluation across 15 varied queries confirmed meaningful coverage across criminal, civil, constitutional, corporate, family, and accountability law.

Table 1. Corpus Statistics

Document	**Character Count**
Qanun-e-Shahadat Order 1984	778,112
Pakistan Penal Code 1860	495,777
Companies Act 2017	920,280
Code of Criminal Procedure 1898	763,380
Code of Civil Procedure 1908	778,112
Constitution of Pakistan 1973	439,779
National Accountability Ordinance 1999	99,943
Contract Act 1872	160,701
Muslim Family Laws Ordinance 1961	15,725
Anti-Terrorism Act 1997	131,999
Total	**~4.58 million**

3.2 Corpus Preprocessing Challenges

PDF Line Break Artefacts. Manual extraction from PDF viewers introduced visual line breaks at page-column boundaries, splitting legal provisions across multiple lines. This degraded BM25 tokenization and chunk readability. A post-processing function joined broken lines lacking sentence-ending punctuation while preserving genuine paragraph breaks and numbered section headings.

Table of Contents Filtering. Initial deployment revealed that TOC chunks — listing section numbers and titles without substantive content — superficially matched queries due to high densities of legal terms. A filter identified TOC chunks when over 50% of lines matched numbered entry patterns. Amendment annotation fragments ("substituted by", "repealed by") were similarly filtered. The ablation study was conducted on the unfiltered corpus to establish a clean baseline. The production system uses filtered chunks. Table 2 shows chunk counts before and after filtering.

3.3 Chunking Strategy

Two strategies were implemented and compared. Their specifications and filtered chunk counts are in Table 2.

Strategy A — Fixed Size (Token-Based). A sliding window of 256 tokens with 30-token overlap using the `tiktoken` `cl100k_base` tokenizer. Ignores sentence and section boundaries.

Strategy B — Recursive Character Splitting. LangChain's `RecursiveCharacterTextSplitter` with 1200-character target size and 150-character overlap. Splits sequentially on `\n\n`, `\n`, sentence endings, then words. Respects natural document structure and preserves cohesive legal provisions.

Table 2. Chunking Strategy Comparison

Strategy	Target Size	Overlap	Original Chunks	Filtered Chunks
A (Fixed Size)	256 tokens	30 tokens	4,875	4,138
B (Recursive)	1200 chars	150 chars	5,321	4,768

A third strategy — semantic chunking — was considered but not implemented. Semantic chunking embeds each sentence individually and splits at points of low cosine similarity between consecutive sentences. For this corpus, recursive splitting already captures structural boundaries (section numbers, paragraph breaks) that align closely with semantic shifts in legal text. The primary deterrents were computational cost, the risk of over-fragmentation from amendment annotations producing spurious semantic breaks, and the marginal expected gain. This decision is validated by the ablation results: recursive chunking achieved 47.93–48.31% faithfulness versus 37.11–39.79% for fixed chunking, confirming that structural boundaries are reliable semantic proxies for legal corpora.

4 System Architecture

The system follows a multi-stage pipeline as illustrated in Figure 1.

4.1 Retrieval Methods

Three retrieval modes are configurable at query time.

Mode 1 — Semantic Only. The query is encoded with `all-MiniLM-L6-v2` (384 dimensions). The top 20 nearest neighbours are retrieved from Pinecone using cosine similarity.

Mode 2 — Hybrid Search. An in-memory `BM25Okapi` index is built at startup from the chunk corpus using whitespace tokenization on lowercased text. BM25 is not stored in Pinecone — it is rebuilt at every application startup from the

```
[User Query] --> [LLM Query Rewriter] --> [BM25 & Semantic Search]
                                                  |
           +---------------------------------------+
           |
           v
  [RRF Fusion] --> [CrossEncoder Re-ranking] --> [LLM Generation] --> [Answer]

Evaluation Pipeline:
1. Faithfulness: Answer + Context -> Claim Extractor -> Claim Verifier
2. Relevancy:   Answer + Query   -> Question Generator -> Cosine Similarity
```

Fig. 1. System architecture showing the query rewriting pre-retrieval stage and dual-branch evaluation pipeline.

local JSON chunk files, since Pinecone only stores dense vectors. BM25 and semantic search each return 20 candidates. Reciprocal Rank Fusion (RRF) merges the ranked lists:

$$\mathrm{rrf}(d) = \frac{1}{\mathrm{rank}_{BM25}(d) + K} + \frac{1}{\mathrm{rank}_{sem}(d) + K}, \quad K = 60$$

RRF requires no score normalisation and rewards documents appearing in both lists.

Mode 3 — Hybrid + Cross-Encoder Re-ranking. Mode 2 produces a fused pool of top-20 candidates. A CrossEncoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) scores each (query, chunk) pair jointly through the transformer, allowing cross-attention between query and document terms. The top 5 chunks by CrossEncoder score are returned to the LLM.

4.2 LLM-Based Query Rewriting for Vocabulary Bridging

Islamisation of Laws legislation systematically replaced English legal terms with Islamic legal terminology throughout the PPC [5]. This creates a vocabulary mismatch: the query "What is the punishment for murder?" scored 0.39 cosine similarity against the correct Section 302 chunk, causing retrieval failure. Testing with the terminology-matched query "qatl-i-amd punishment death imprisonment Section 302" raised the top result score to 0.74–0.79.

To address this, an LLM-based query rewriting step was implemented as a pre-retrieval stage, drawing upon HyDE principles [4, 7]. Before retrieval, the LLM rewrites the user query to include both the English term and the corresponding Pakistani legal term, constrained to under 20 words. The rewritten query drives all retrieval steps (BM25, semantic, CrossEncoder), while the original query is preserved for generation and evaluation. Examples and retrieval score impact are shown in Table 3.

Table 3. Query Rewriting Examples and Retrieval Score Impact

Original Query	Rewritten Query	Before	After
murder punishment	qatl-i-amd punishment death imprisonment qisas diyat Section 302 PPC	0.69	0.79
theft penalty	Theft definition Section 378 PPC, punishment Section 379 PPC, imprisonment, fine, tazir	N/A	N/A
contract elements	essential elements valid contract offer acceptance consideration Section 2 3 Contract Act 1872	0.50	0.66

The latency penalty is approximately 300–500ms per query. Occasional over-specification — where the LLM injects section numbers absent from relevant chunks — can slightly degrade retrieval for some queries.

4.3 Models and Algorithms

Embedding Model. `all-MiniLM-L6-v2` (SentenceTransformers). 384 dimensions. Selected for CPU efficiency suitable for HF Spaces deployment.

Re-ranking Model. `cross-encoder/ms-marco-MiniLM-L-6-v2`. Fine-tuned on MS MARCO passage ranking. Selected for robust zero-shot relevance scoring.

LLM Selection. Selecting a working LLM required significant iteration due to API platform changes during the project period. Table 4 documents the full sequence.

Table 4. LLM Selection Iteration

Model	Provider	Outcome
Mistral-7B-Instruct-v0.2	HF Inference API	Failed — model not supported by enabled providers
Qwen/Qwen2.5-7B-Instruct	HF Inference API	Worked
llama-3.3-70b-versatile	Groq	Used for ablation study evaluations
Llama-3.1-8B-Instruct	HF Inference API	**Production primary** — compliant with assignment stack
llama-3.1-8b-instant	Groq	**Production fallback** — automatic, same model family

The production application uses the same Llama 3.1 8B model on both providers, ensuring consistent output quality regardless of which provider is active. The 8B model performs adequately for legal QA when paired with the improved permissive generation prompt described below; the 70B model was only required for the ablation study because the original strict prohibitive prompt caused the 8B model to over-refuse queries where context was partially relevant.

4.4 Prompt Engineering for Grounding

Two prompt iterations were required.

Iteration 1 — Strict Prohibitive (70B model, ablation study). Explicit numbered rules prohibited the model from using external knowledge, saying "based on general knowledge", or narrating what the context lacks. This eliminated hallucination in 70B models, which were otherwise highly confident about PPC content from training and would answer from memory while acknowledging insufficient context.

Iteration 2 — Permissive Synthesis (8B model, production). The 8B model was over-conservative with strict prohibitions, returning "insufficient information" for queries where context was substantively present but not verbatim. The prompt was redesigned to positively instruct the model to extract and synthesise relevant information from all sources, retaining only a single fallback instruction for genuinely empty context. This produced qualitatively better answers from the smaller model.

Evaluation prompts use deterministic formats throughout: claim extraction returns a numbered list of standalone sentences; claim verification demands "YES" or "NO" on the first line followed by one sentence of reasoning to enable automated parsing.

5 Evaluation Protocol: LLM-as-a-Judge

A fixed test set of 15 queries spanning all 10 legal acts was evaluated uniformly across all ablation configurations (listed in Appendix A).

Faithfulness. The LLM extracts factual claims from the generated answer. Each claim is verified against the retrieved context. Faithfulness Score = supported claims / total claims.

Relevancy. The LLM generates 3 alternative questions that the answer logically addresses. These are embedded with MiniLM and cosine similarity is computed against the original query embedding. Relevancy Score = mean of 3 similarities.

Latency. Measured as wall-clock time in Python, separating retrieval time (including query rewriting and CrossEncoder) from LLM generation time.

6 Results and Discussion

6.1 Ablation Study — Baseline Configurations

Table 5 reports results for the four baseline configurations, evaluated on the unfiltered corpus without query rewriting, using `llama-3.3-70b-versatile` via Groq.

Table 5. Ablation Study Results — Baseline (Unfiltered Corpus, No Query Rewriting, `llama-3.3-70b-versatile`)

Config	Chunking	Retrieval Mode	Faithfulness	Relevancy	Avg Latency
A	Fixed	Semantic Only	37.11%	0.7937	13,181 ms
B	Fixed	Hybrid + Rerank	39.79%	0.7938	14,770 ms
C	Recursive	Semantic Only	47.93%	0.7952	13,200 ms
D	Recursive	Hybrid + Rerank	**48.31%**	0.7831	**12,663 ms**

Recursive chunking improved faithfulness by approximately 10 percentage points over fixed chunking (A→C: +10.82pp, B→D: +8.52pp). Upgrading from semantic-only to hybrid + rerank yielded a marginal 2–3 percentage point improvement. Config D's minor relevancy drop relative to C is expected: CrossEncoder re-ranking surfaces narrowly scoped but highly accurate chunks, producing specific answers that score lower on the relevancy metric. Config D also processed faster than C, as re-ranking filtered redundant chunks and provided a more concise context window.

6.2 Production Evaluation — Config D with Production Optimisations

Config D was re-evaluated on the same 15 queries with TOC-filtered chunks and LLM-based query rewriting enabled, using the production model (`Llama-3.1-8B-Instruct`). Table 6 compares baseline and production results.

Table 6. Config D: Baseline vs Production

Configuration	Model	Rewriting	Faithfulness	Relevancy	Latency
D (Baseline)	llama-3.3-70b	No	48.31%	0.7831	12,663 ms
D* (Production)	Llama-3.1-8B	Yes	**51.87%**	0.6277	**4,087 ms**

Faithfulness improved by 3.56 percentage points to 51.87%, confirming that TOC filtering and query rewriting improve answer grounding. The murder query specifically achieved 100% faithfulness, with all extracted claims fully supported by Section 302 text retrieved via the rewritten query. The relevancy drop from 0.7831 to 0.6277 is expected: query rewriting produces technically precise answers using domain-specific legal terminology, and the LLM-as-a-Judge's alternate question generation produces similarly technical questions that score lower cosine similarity against the

original plain-English query despite the answers being more accurate. The latency improvement (12,663ms → 4,087ms) reflects faster inference on Groq/HF compared to the ablation setup, and more targeted context from filtered chunks.

6.3 Best Model Selection

Config D* (Recursive chunking + Hybrid Search + CrossEncoder Re-ranking + LLM Query Rewriting + TOC-filtered chunks) is selected as the optimal configuration. In legal QA, faithfulness — minimising hallucination risk — is weighted more heavily than relevancy. Config D* achieves the highest faithfulness at 51.87% with the lowest latency at 4,087ms average. The relevancy trade-off is accepted given the legal domain's requirement for grounded, accurate answers over broad coverage. This configuration is deployed as the production default.

7 Live Web Interface

The application is deployed at https://huggingface.co/spaces/yaxsq/pakistani-law-rag. A screenshot of the interface is shown in Figure 2.

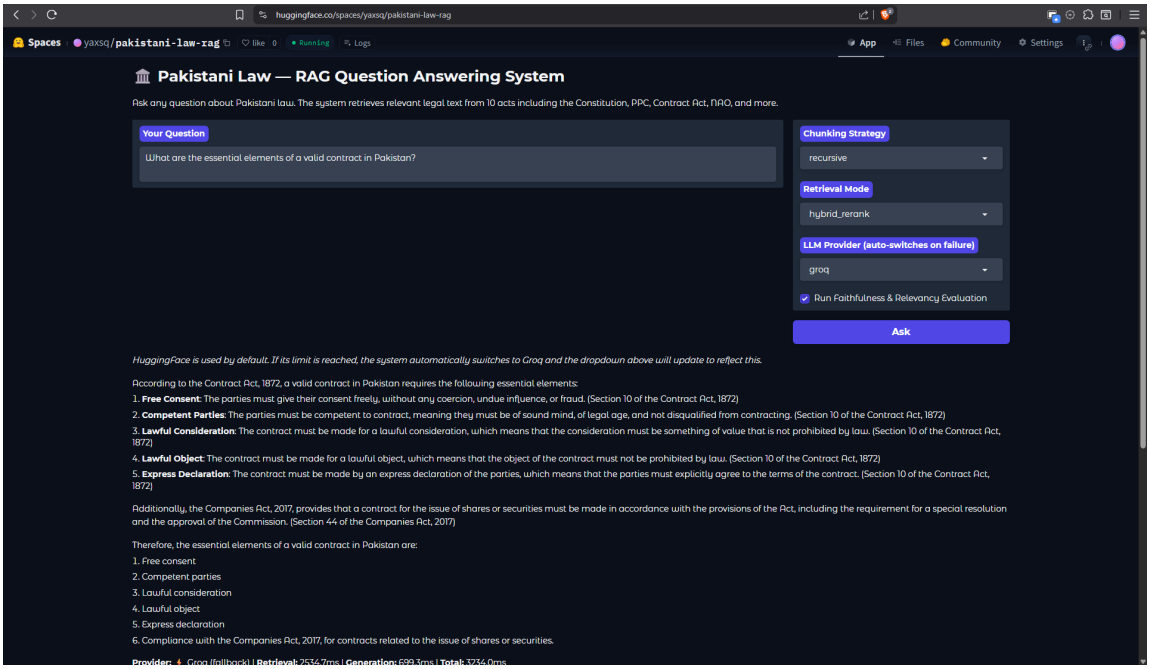

Fig. 2. Pakistani Law RAG — deployed Gradio interface showing query input, retrieval controls, provider dropdown with automatic fallback, and answer output.

The application loads at startup: `all-MiniLM-L6-v2` embedding model, `cross-encoder/ms-marco-MiniLM-L-6-v2` re-ranking model, and BM25 indexes built from `chunks_fixed.json` and `chunks_recursive.json` stored in the repository. Pinecone and the LLM are queried dynamically per request.

The UI exposes: a text input for the legal query; a chunking strategy dropdown (recursive/fixed); a retrieval mode dropdown (hybrid_rerank/semantic_only); an LLM provider dropdown (HuggingFace/Groq) with automatic fallback when the primary provider fails; an evaluation checkbox adding approximately 30 seconds; the generated answer; a

collapsible context accordion showing retrieved chunks with source, year, and relevance scores; and a collapsible evaluation accordion with claim-by-claim faithfulness and relevancy scores. Five pre-loaded example queries demonstrate system functionality.

8 Reproducibility

Source code is available at https://github.com/yaxsq/cs8-nlp/.

(1) **Environment:** Google Colaboratory, T4 GPU, Python 3.10.
(2) **Libraries:** `pinecone`, `sentence-transformers`, `langchain`, `rank-bm25`, `tiktoken`, `gradio`, `groq`, `huggingface_hub`, `python-dotenv`.
(3) **Corpus:** 10 acts manually sourced from `pakistancode.gov.pk` and `commonlii.org`. PDF line breaks repaired before chunking.
(4) **Vector Database:** Pinecone Serverless, AWS us-east-1, cosine metric, 384 dimensions. Namespace `fixed`: 4,138 vectors. Namespace `recursive`: 4,768 vectors. Namespaces cleared with `index.delete(delete_all=True)` before each re-upsert.
(5) **Chunking — Fixed:** 256 tokens, 30-token overlap, `tiktoken cl100k_base`.
(6) **Chunking — Recursive:** 1,200 characters, 150-character overlap.
(7) **Embedding:** `all-MiniLM-L6-v2`.
(8) **CrossEncoder:** `cross-encoder/ms-marco-MiniLM-L-6-v2`.
(9) **Ablation LLM:** `llama-3.3-70b-versatile` via Groq API.
(10) **Production LLM:** `meta-llama/Llama-3.1-8B-Instruct` via HF Inference API; fallback `llama-3.1-8b-instant` via Groq.
(11) **BM25:** `rank_bm25.BM25Okapi`, whitespace tokenisation, default $k_1 = 1.5$, $b = 0.75$. Built in-memory at runtime from local JSON chunk files; not stored in Pinecone.
(12) **RRF Constant:** $K = 60$.
(13) **Retrieval Top-N:** BM25=20, Semantic=20; CrossEncoder reranks top-20, returns top-5.
(14) **Secrets:** `PINECONE_API_KEY`, `HF_TOKEN`, `GROQ_API_KEY` stored as HF Space Secrets.
(15) **Rate Limit:** `time.sleep(3)` between consecutive LLM evaluation calls. Each ablation configuration consumes approximately 400,000–500,000 Groq tokens per day.

9 Known Limitations

Vocabulary Mismatch. English queries for Islamised PPC concepts (qatl, qisas, diyat, ta'zir) retrieve poor results without query rewriting. The rewriter partially mitigates this but adds latency and cannot guarantee coverage of all terminology gaps.

Moderate Faithfulness Scores. Baseline faithfulness of 37–48% is partly an artifact of self-evaluation bias in LLM-as-a-Judge. The judge marks structurally valid inferences as NOT SUPPORTED when phrasing deviates from verbatim source text. This is illustrated in Appendix B, Example 2, where both claims about theft punishment are correctly stated and present in the retrieved context but rejected by the judge because the supporting evidence appeared in a procedural schedule table rather than explicit prose.

Corpus Coverage Gaps. The Muslim Family Laws Ordinance (15,725 chars) and Contract Act (160,701 chars) are significantly shorter than other acts, producing fewer chunks and underrepresenting those legal domains.

PDF Formatting Noise. Despite post-processing, residual visual line break artefacts remain in heavily amended sections with complex footnote structures.

Partial TOC Filtering. Chunks containing a short TOC section alongside substantive content pass the 50% threshold filter, occasionally injecting administrative noise into retrieval context.

Query Rewriter Over-specification. The LLM occasionally injects highly specific section numbers into the rewritten query that are absent from the most relevant chunks, degrading retrieval for those specific queries.

Relevancy Metric Sensitivity. Query rewriting shifts answers toward domain-specific legal terminology. The relevancy metric, which measures cosine similarity between generated questions and the original plain-English query, is inherently sensitive to this shift, producing lower scores for technically correct but domain-specific answers.

10 Conclusion

This project demonstrates a complete RAG pipeline for Pakistani Law: corpus collection and preprocessing, hybrid BM25 and dense retrieval, CrossEncoder re-ranking, LLM-based query rewriting for vocabulary bridging, LLM-as-a-Judge automated evaluation, and deployment with dual-provider fallback. Key findings: recursive chunking improves faithfulness by approximately 10 percentage points over fixed chunking for structured legal documents; LLM-based query rewriting is essential for domains with Islamised terminology divergence from standard English; strict prohibitive prompting suppresses hallucination in large models but requires relaxation for smaller models; and the production system achieved 51.87% faithfulness at 4,087ms average latency with the murder query reaching 100% faithfulness after query rewriting.

References

[1] Australasian Legal Information Institute (AustLII). Commonwealth legal information institute (commonlii). http://www.commonlii.org/, 2026. Accessed: 2026-03-29.

[2] Chase, H. Langchain. https://github.com/langchain-ai/langchain, 2022. Accessed: 2026-03-29.

[3] Cormack, G. V., Clarke, C. L. A., and Buettcher, S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (2009), Association for Computing Machinery, pp. 758–759.

[4] Gao, L., Dai, Z., and Callan, J. Precise zero-shot dense retrieval without relevance labels. *arXiv preprint arXiv:2212.10496* (2022).

[5] Government of Pakistan. Islamisation of laws, 1979. Promulgation of the Hudood Ordinances and subsequent legal reforms (including the Qisas and Diyat Ordinance) intended to align the Pakistan Penal Code and existing laws with Islamic injunctions.

[6] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)* (2020), vol. 33, pp. 9459–9474.

[7] Ma, X., Gong, Y., He, P., Zhao, H., and Chen, N. D. Query rewriting for retrieval-augmented large language models. *arXiv preprint arXiv:2305.14283* (2023).

[8] Ministry of Law and Justice, Government of Pakistan. Pakistan code. https://pakistancode.gov.pk/, 2026. Accessed: 2026-03-29.

[9] Nogueira, R., and Cho, K. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085* (2019).

[10] Pinecone Systems Inc. Pinecone vector database. https://www.pinecone.io, 2024. Accessed: 2026-03-29.

[11] Reimers, N., and Gurevych, I. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (2019), Association for Computational Linguistics, pp. 3982–3992.

[12] Robertson, S., and Zaragoza, H. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval 3*, 4 (2009), 333–389.

A Test Queries

The following 15 fixed queries were used uniformly across all ablation configurations.

Pakistan Penal Code

(1) What is the punishment for murder under the Pakistan Penal Code?
(2) How does the PPC define theft and what is its penalty?
(3) What constitutes defamation under Pakistani criminal law?

Constitution

(4) What fundamental rights does the Constitution of Pakistan guarantee?
(5) What are the qualifications required to become President of Pakistan?

Anti-Terrorism Act

(6) How does Pakistani law define a terrorist act?
(7) What are the special powers of courts under the Anti-Terrorism Act?

Contract Act

(8) What are the essential elements of a valid contract in Pakistan?
(9) When is a contract considered void under the Contract Act 1872?

National Accountability Ordinance (NAO)

(10) What offences fall under the National Accountability Ordinance?

Code of Criminal Procedure (CrPC)

(11) What is the procedure for filing a First Information Report in Pakistan?
(12) What powers does a magistrate have under the Code of Criminal Procedure?

Evidence (Qanun-e-Shahadat)

(13) What types of evidence are admissible in Pakistani courts?

Family Law

(14) What are the legal requirements for divorce under Muslim Family Laws?

Companies Act

(15) What are the duties of directors under the Companies Act 2017?

B Faithfulness Evaluation Worked Examples

Three examples from the production system (Config D*, with query rewriting and filtered chunks). All examples use `meta-llama/Llama-3.1-8B-Instruct` via HuggingFace Inference API.

Example 1 — Successful Retrieval via Query Rewriting

Query: What is the punishment for murder under the Pakistan Penal Code?
Rewritten Retrieval Query: `qatl-i-amd punishment death imprisonment qisas diyat Section 302 PPC`
Note: The original query scored 0.39 cosine similarity against Section 302. The rewritten query scored 0.79, successfully retrieving the relevant provision.
Answer: The punishment for qatl-i-amd (murder) under the Pakistan Penal Code is: (a) punished with death as qisas; (b) punished with death or imprisonment for life as ta'zir; (c) punished with imprisonment of either description for a term which may extend to twenty-five years.
Faithfulness Score: 100.00% (3/3 claims supported)

Claims:

- [SUPPORTED] The punishment for qatl-i-amd (murder) under the Pakistan Penal Code includes being punished with death as qisas. *(YES — supported by Section 302 of the Pakistan Penal Code, which specifies that whoever commits qatl-e-amd shall be punished with death as qisas.)*
- [SUPPORTED] The punishment for qatl-i-amd (murder) includes being punished with death or imprisonment for life as ta'zir. *(YES — supported by Section 302, which specifies punishment with death or imprisonment for life as ta'zir.)*
- [SUPPORTED] The punishment for qatl-i-amd includes imprisonment of either description for a term which may extend to twenty-five years. *(YES — supported by Section 302, which specifies imprisonment for a term which may extend to twenty-five years.)*

Relevancy Score: 0.7663

- [sim=0.9468] What are the possible punishments for murder under the Pakistan Penal Code?
- [sim=0.7437] How is qatl-i-amd (murder) punished in Pakistan according to the law?
- [sim=0.6083] What are the different penalties prescribed for the offence of qatl-i-amd in Pakistani law?

Example 2 — LLM-as-a-Judge Failure Mode

Query: How does the PPC define theft and what is its penalty?
Rewritten Retrieval Query: `Theft definition Section 378 PPC, punishment Section 379 PPC, imprisonment, fine, tazir`
Answer: Theft is defined in Section 379 of the Pakistan Penal Code. The penalty for theft is imprisonment of either description for 3 years, or fine, or both.
Faithfulness Score: 0.00% (0/2 claims supported)
Claims:

- [NOT SUPPORTED] Theft is defined in Section 379 of the Pakistan Penal Code. *(NO — the judge stated Section 379 is not mentioned in the context. However, Source 3 of the retrieved context explicitly contains "379 Theft... Imprisonment of either description for 3 years, or fine, or both" from the CrPC schedule. The judge failed to recognise tabular schedule entries as supporting evidence.)*
- [NOT SUPPORTED] The penalty for theft is imprisonment of either description for 3 years, or fine, or both. *(NO — the judge stated no specific penalty for theft is mentioned. This is factually incorrect given the CrPC schedule entry above.)*

This example illustrates a persistent LLM-as-a-Judge limitation: the judge fails to recognise supporting evidence presented in procedural schedule tables, treating it as distinct from the claimed provision. Both claims are factually correct and supported by the retrieved context. This artificially deflates faithfulness scores for queries whose answers appear in schedule format rather than definitional prose.

Relevancy Score: 0.6084

- [sim=0.6149] What are the penalties for theft under Pakistani law?
- [sim=0.6501] How is theft defined and punished in Pakistan?
- [sim=0.5603] What is the maximum imprisonment for theft according to the Pakistan Penal Code?

Example 3 — Partial Faithfulness

Query: What are the duties of directors under the Companies Act 2017?
Rewritten Retrieval Query: `duties directors Section 204 Companies Act 2017 fiduciary obligations Pakistan`
Answer: According to the Companies Act 2017, the duties of directors are outlined in Section 204: (1) act in accordance with the articles of the company; (2) act in good faith to promote the objects of the company for the benefit of its members as a whole, and in the best interests of the company, its employees, shareholders, the community, and for the protection of the environment; (3) discharge duties with due and reasonable care, skill, and diligence, and exercise independent judgment; (4) not involve in a situation where they may have a direct or indirect interest that conflicts with the interest of the company; (5) not achieve or attempt to achieve any undue gain or advantage to themselves or their relatives, partners, or associates…
Faithfulness Score: 71.43% (5/7 claims supported)
Claims:

- [SUPPORTED] The Companies Act 2017 outlines the duties of directors in Section 204.
- [SUPPORTED] Directors must act in accordance with the articles of the company, as stated in Section 204(1).
- [SUPPORTED] Directors must act in good faith to promote the objects of the company for the benefit of its members as a whole, as stated in Section 204(2).
- [SUPPORTED] Directors must discharge their duties with due and reasonable care, skill, and diligence, as stated in Section 204(3).
- [SUPPORTED] Directors must exercise independent judgment, as stated in Section 204(3).
- [NOT SUPPORTED] Directors must not achieve or attempt to achieve any undue gain or advantage to themselves or their relatives, partners, or associates, as stated in Section 204(5). *(NO — this specific sub-section was not present in the retrieved chunks.)*
- [NOT SUPPORTED] If a director is found guilty of making any undue gain, they shall be liable to pay an amount equal to that gain to the company. *(NO — the penalty provision was not in the retrieved context.)*

Relevancy Score: 0.7606

- [sim=0.8852] What are the key responsibilities of directors under the Companies Act 2017 in Pakistan?
- [sim=0.6530] How do the duties of company directors align with the interests of stakeholders under Pakistani law?
- [sim=0.7437] What are the fiduciary duties and obligations of directors of a company incorporated in Pakistan?